

Date _____
Page No. _____

Name: Manu Sharan Kumar

Roll No: ACE080BCT037

Subject: Software Engineering

Assignment-III

1. What is the importance of the SRS document? List and explain functional (any two) and non-functional requirements of an airline reservation system.

Ans A Software Requirements Specification (SRS) is a formal document that describes the behaviour, features, and constraints of a system to be developed. Its primary importance includes:

- Agreement Baseline

SRS serves as a binding contract between customers and developers regarding what the software will deliver.

- Reduction of Rework:

SRS clarifies requirements clearly, preventing costly modifications late in the development lifecycles.

- Basis for Estimation & Planning:

SRS provides the accurate scope needed to estimate costs, project schedules, and resource allocation.

- Foundation for Testing:

Validation and verification team uses SRS to write test cases and ensure the final product matches expectations.

Ans Any two functional requirements of airline reservation system are:

1) Flight Search & Booking:

The system must allow users to search for available flights by origin, destination, date, and class, and proceed to select and book seats.

2) e-Ticket Generation & PNR Management

The system must generate a unique Passenger Name Record (PNR) and issue a downloadable e-ticket upon successful payment confirmation.

Ans Any two non-functional requirements of airline reservation system are:

1) Performance (Response Time):

Flight search queries must return within 2 seconds for upto 10,000 concurrent users.

2) Security & Compliance:

All payments transactions must adhere to PCI-DSS standards, utilizing 256-bit^{SSL} encryption for data in transit.

2. Distinguish between user and system requirements. Why does an engineer ensure that functional and non-functional needs are in a requirement specification document?

Ans The differences between user and system requirements are:

User Requirements

1) They are high-level statements of what services the system should provide in natural language.

2) The target audience of it are non-technical stakeholders, end-users, management, and project sponsors.

3) It has simple natural language, basic diagrams, and user stories.

4) For example:

The system shall allow passengers to view their flight status.

System Requirements

They are detailed functional and operational specifications of the software system.

The target audience of it are software developers, architects, system engineers, and testers.

It has structured specifications, detailed domain models, exact input/output definitions.

For example:

When GET /flight-status/{id} is invoked, the system shall query the operational database and return JSON containing gates, delays, and status within 500ms.

Ans An engineer must ensure that functional and non-functional needs are in a requirement specification document because of

① Completeness:

Functional requirements state whether what the system does; non-functional requirements state how

well it performs those functions (e.g., speed, security, scalability).

⑥ Architectural Guidance:

Non-functional requirements directly influence system architecture (e.g., choosing microservices over monoliths to meet extreme availability targets). A system that satisfies all functional goals but crashes under real-world load is a failure.

3. Explain the requirement engineering process in detail.

Ans The Requirement Engineering (RE) process consists of five structured phases as:

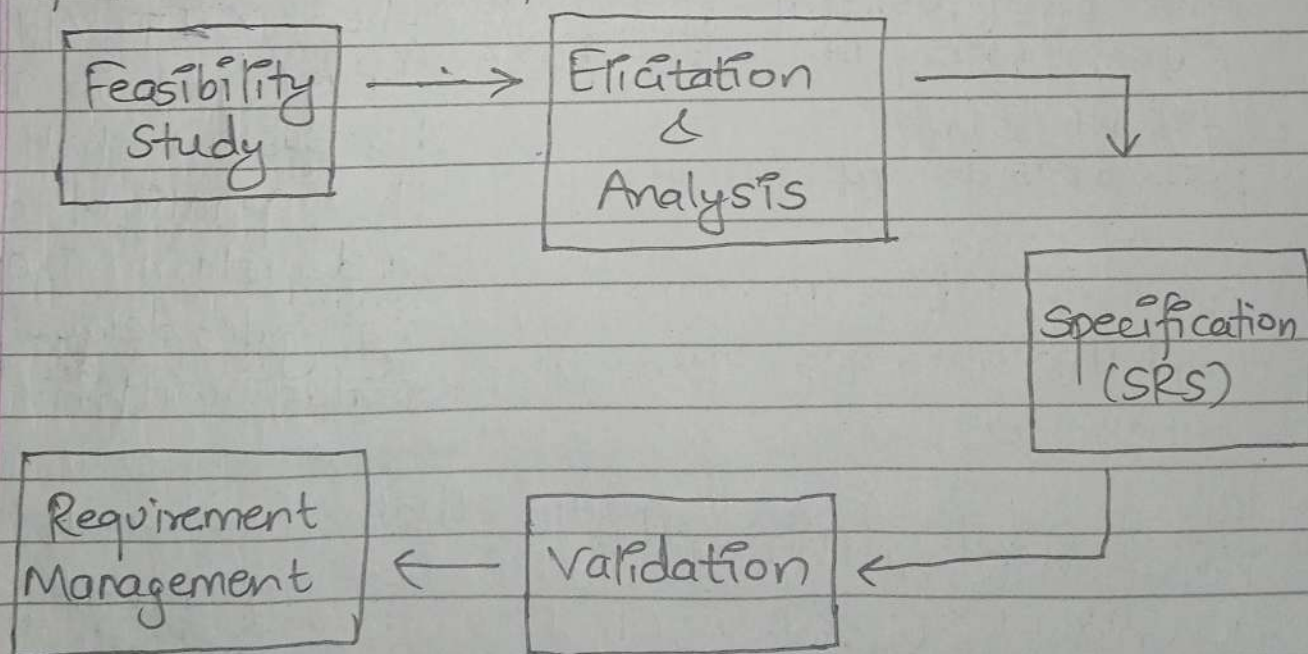


Fig.: Requirement Engineering Process

Date _____
Page No. _____

② Feasibility Study:

Assesses whether the proposed software is technically achievable, economically viable, and operationally realistic within the given budget and timeline.

③ Requirements Elicitation & Analysis:

Requirements are gathered from stakeholders through interviews, workshops, surveys, observation, and prototyping. Analysts resolve conflicts and categorize requirements.

④ Requirements Specification

The gathered information is formatted into a formal Software Requirements Specification (SRS) document using standardized templates (e.g., IEEE 830/ISO/IEC/IEEE 29148).

⑤ Requirements Validation

Ensures the specified requirements are complete, consistent, unambiguous, and realistic. Reviews, walkthroughs, and inspections are conducted with stakeholders.

⑥ Requirements Management

Manages changes to requirements over time through change-control procedures, maintaining traceability matrices throughout the software development lifecycle.

4. For the telemedicine project, (i) list out all functional and non-functional requirements, and (ii) Make a project Feasibility Report.

Ans The functional requirements for a telemedicine project are:

- Video Construction:
Enables high-definition encrypted video/audio calls between patients and licensed doctors.
- Appointment Scheduling:
Allows patients to browse doctor availability, book appointments, and receive automated SMS/email reminders.
- E-Prescriptions:
Enables doctors to issue digital prescriptions directly sent to the patient's preferred pharmacy.
- Electronic Health Record (EHR) Access:
Provides secure access for patients and clinicians to view medical histories, lab results, and diagnostic images.

Ans The non-functional requirements for a telemedicine project are:

- Privacy & Compliance:
Must fully comply with HIPAA/GDPR regulations regarding patient health information (PHI).
- Reliability:
Video stream latency must remain under 150ms during low-bandwidth connections (minimum 512 Kbps).

- Availability:

System services must remain 99.95% uptime (24/7 operation)

Project Feasibility Report:

The proposed Telemedicine Platform connects remote patients with medical practitioners for online triage, consultation, and digital prescriptions.

Feasibility Dimensions:

- Technical Feasibility: FEASIBLE

Leverages established WebRTC protocol for real-time video streaming, cloud-hosted microservices (AWS/Azure), and TLS 1.3 encryption.

- Economic Feasibility: FEASIBLE

Estimated initial development cost is \$15,000. Projected operational savings (reduced clinic overhead, expanded patient reach) predict a Return on Investment (ROI) within 18 months.

- Operational Feasibility: HIGH

Doctors and patients use intuitive interfaces (web/mobile). Minimum user training is required due to standard mobile UI design patterns.

- Legal & Regulatory Feasibility: FEASIBLE

Data retention and user consent modules will be built to comply with health data laws (HIPAA/GDPR).

5. Why is it difficult to gain a clear understanding of what the customer wants? What are the guidelines for the requirement elicitation process?

Ans It is difficult to gain a clear understanding of what the customer wants because of

(a) Tacit Knowledge:

Users know their job well but struggle to explain step-by-step routines ("it's just second nature")

(b) Evolving Requirements:

Stakeholders often do not know what they want until they see a working prototype.

(c) Domain Vocabulary:

Communication barriers exist between technical jargon (developers) and domain terminology (business users).

(d) Conflicting Stakeholder Interests:

Different user groups (e.g. management vs operational staff) often request contradictory features.

Ans The guidelines for the requirement elicitation process are:

(a) Use Contextual Inquiry:

Observe end-users performing their daily tasks in their actual environment rather than relying only on Q&A sessions.

⑥ Build Rapid Prototypes:

Creates wireframes or interactive mockups early to give stakeholders a concrete artifact to react to.

⑦ Define a Shared Glossary:

Establish a common domain vocabulary dictionary to avoid misinterpretations.

⑧ Focus on Goals, Not Solutions:

Ask stakeholders what problem they need solved, rather than asking them how to code the solution.

6. What is the purpose of going through a feasibility study? List out the various types of feasibility and explain any two types in detail.

Ans A feasibility study is conducted prior to contract signing or development to evaluate whether the project is practical, cost-effective, and achievable given technical and organizational constraints.

Ans The various types of feasibility are:

- ① Technical Feasibility
- ② Economic Feasibility
- ③ Operational Feasibility
- ④ Schedule Feasibility
- ⑤ Legal & Compliance Feasibility

- Technical Feasibility

Evaluates whether the team possesses (or can acquire) the hardware, software tools, network infrastructure, and engineering skills required to build the ~~inf~~ proposed system.

Key Assessment Areas: Hardware limitations, tech stack maturity, availability of skilled personnel, integration constraints with legacy systems.

- Economic Feasibility

Conducts a Cost-Benefit Analysis (CBA) to evaluate whether the financial return of the completed software outweighs the capital expenditure required to develop and maintain it.

Key Assessment Areas: Development cost, licensing fees, operational cost vs. expected revenue, ROI timeline, payback period.

7. Why is architectural design important? Explain client-server architecture with a suitable example.

Ans Architectural design defines the high-level structure of a system, breaking it into interacting subsystems. It is critical because:

- Controls Quality Attributes:

Non-functional characteristics like performance, fault tolerance, maintainability, and security are

determined by the architecture.

- Facilitates stakeholder communication

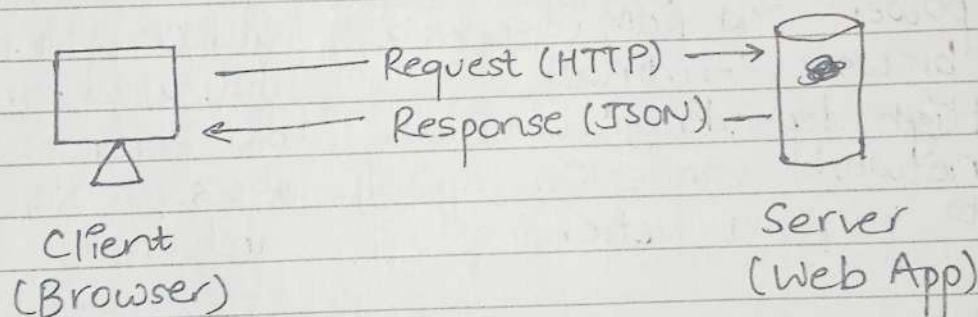
High-level abstractions can be understood by non-technical managers and technical architects alike.

- Enables Module Reuse:

Clear boundaries allow subsystems to be reused across different enterprise applications.

Client-Server Architecture

A distributed structure where workloads are partitioned between providers of a resource or service (servers) and service requesters (clients).



- Client: Initiates requests, manages user interaction/UI.
- Server: Processes incoming requests, execute business logic, manages persistent storage, and sends responses.

Eg: A web browser accessing Wikipedia. The browser (client) sends an HTTP GET request over the network, and Wikipedia's web server processes

Date _____
Page No. _____

the request, retrieves data from its databases, and returns HTML/CSS back to the client.

8. Differentiate between Thin and Fat client model client-server architecture.

Ans The differences between Thin client Architecture and Fat (Thick) Client Architecture are:

Thin Client Architecture	Fat Client Architecture
1) Almost all application and business logic runs on the central server.	Application and business logic execute locally on the client machine.
2) Requires minimal processing power and RAM (e.g. web browser, terminal)	Requires powerful client hardware (CPU, RAM, dedicated graphics)
3) High dependency; continuous network connection required to perform actions.	Low dependency; can process data offline and synchronize with the server later.
4) Deployments & updates are easy; updating the application on the server updates it instantly for all clients.	Deployments & updates are complex; updates must be installed on every client device individually.

9. Differentiate between the repository model and layered model with their advantages and disadvantages.

Ans The differences between the repository model and layered model are:

Repository Architectural Model

Layered Architectural Model

- | | |
|---|---|
| ① Subsystems share data via a central, common database/repository. | Organizes system into a vertical stack of layers, each providing services to the layer above. |
| ② All components pass data through the central store. | Data flows sequentially up and down through the adjacent layers. |
| ③ Components are decoupled from each other, but tightly coupled to the central store. | High cohesion within layers; loose coupling between non-adjacent layers. |

Repository Model.

- Advantages

- Efficient for sharing large amounts of data across multiple tools.

- Easy to integrate new subsystems without affecting existing ones (subsystems do not need to know about each other)

- Disadvantages

- Central repository is a single point of failure and bottleneck for performance.

Date _____
Page No. _____

→ Translating all data to a single schema across diverse tools can be difficult and inefficient.

Layered Model

- Advantages

- High maintainability; replacing a layer only requires matching its existing interface.
- Enhances security and abstraction since upper layers cannot layer directly access low-level drivers.

- Disadvantages:

- Performance overhead due to data traversing multiple intermediate boundaries.
- Clean separation into layers can be difficult to achieve for complex real-world logic.

10. What is system architecture? Explain its importance. Explain the pipe and filter architecture and call-return architecture in brief.

Ans System architecture is the conceptual model that defines the structure, behaviour, key components, and relationships of a sys. software system.

The importance of system architecture are:

- Sets the structural blueprint for multi-team parallel development.
- Directs trade-off analysis (e.g., sacrificing raw speed for higher security).

→ Reduces future maintenance costs by enforcing structural organization.

1) Pipe and Filter Architecture.

Processes continuous streams of data. The system consists of Filters (processing components) connected by Pipes (data channel). Each filter transforms incoming data and passes the output to the next filter.

Eg: Unix shell commands

(cat file.txt | grep "ERROR" | wc -l).

2) Call-and-Return Architecture.

A classic hierarchical control structure where a main program invokes subroutines/methods in a deterministic sequence. Control passes down the invocation tree and returns back to the calling routine once execution completes.

Eg: Traditional procedural programs

(e.g., main() calling helper functions in C/Java).

11. What are the different modular decomposition styles used during system design? What is the repository model? Explain its advantages and disadvantages.

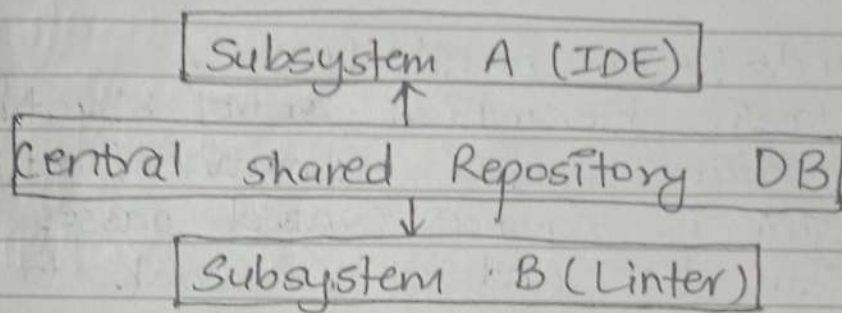
Ans

The modular decomposition styles break down a system into smaller modules. Common styles include:

① Object-oriented Decomposition:
Systems are decomposed into autonomous objects that encapsulate both data structures and state-changing operations (methods).

② Function-Oriented / Data-Flow Decomposition:
Systems are structural around functional transformations where data flows through processing functions.

The Repository Model:



A design pattern where all system components communicate exclusively through a central idea data store.

• Advantages:

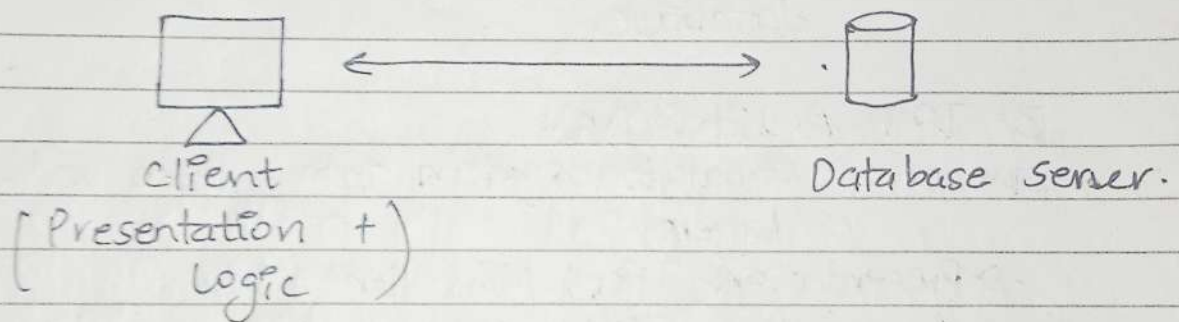
- Subsystems operate independently without managing direct communication links with one another.
- Data backup, recovery, and access control are centralized at one point.

- Disadvantages

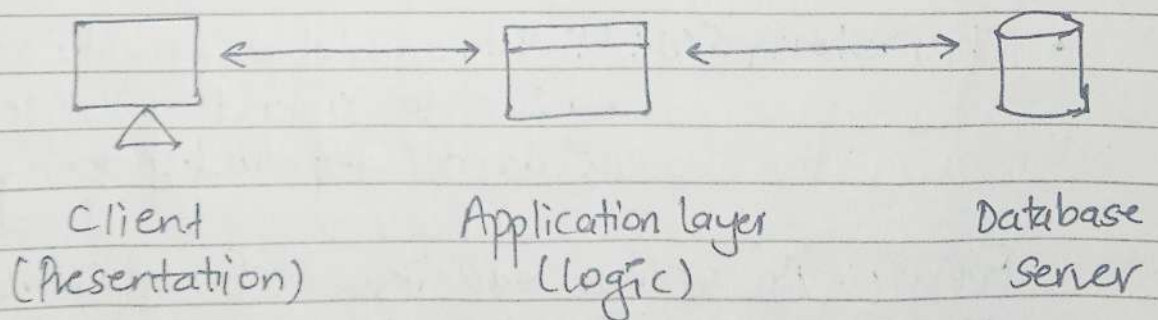
- The central repository can become an operational performance bottleneck.
- Evolution of the central database schema requires updating all dependent components.

12. Explain 2-tier and 3-tier architecture with an example.

Ans 2-Tier Architecture



3-Tier Architecture



2-Tier Architecture:

- **Structure:** Consists of two layers. Client (UI and application/business logic) directly interfacing with a Database server.
- **Characteristics:** Simple design, but difficult to scale; business rules are embedded directly into client applications or database stored procedures.
- **Example:** A desktop-based inventory application installed directly on a warehouse manager's PC connecting to a local SQL database.

3-Tier Architecture :

- **Structure:** Separates system into three independent layers
 - **Presentation Tier:** UI/client layer
 - **Application Tier:** Core business rules and logic processing server.
 - **Data Tier:** Central database management system (DBMS).
- **Characteristics:** Highly scalable, modular, secure, and easier to upload business logic without redeploying client software.

For eg': An online banking web application where a user browser (Tier 1) connects to an enterprise Java (Node.js) app server (Tier 2), which queries an

Oracle / PostgreSQL DB (Tier-3)

13. Explain why it may be necessary to design the system architecture before specifications are written? Explain in detail about distributed object architecture with a suitable example.

Ans It can be necessary to design the system architecture before finalizing complete requirement specifications because:

- (a) COTS / Component Integration:

Enables selection and evaluation of commercial-off-the-shelf (COTS) software and existing large legacy platforms that constrain overall requirements.

- (b) Early Cost & Viability Analysis:

Architecture provides the structural baseline required to assess whether non-functional demands (e.g., sub-millisecond response times) are financially and technically viable before writing exhaustive specs.

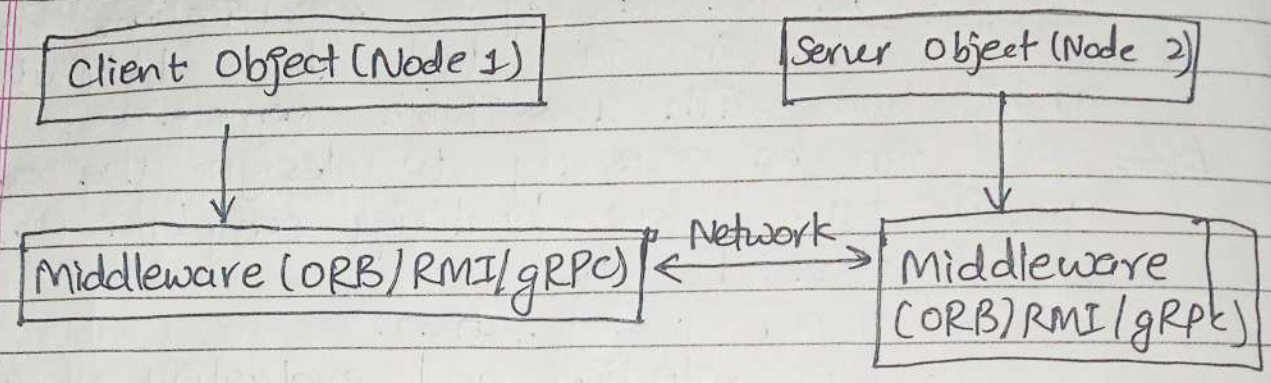
- (c) Contracting & Subsystem Delegation:

High-level architectural boundaries allow different sub-contractors or internal engineering teams to write specifications for their respective subsystems in parallel.

Distributed Object Architecture

In a Distributed Object Architecture, system components are implemented as independent objects

residing on separate machines connected across a network. Objects communicate through remote procedure invocations via an Object Request Broker (ORB) or Middleware Layer.



Key Characteristics:

- Location Transparency:
A client object calls methods on remote objects using the same syntax as local method calls; the middleware handles network transport.
- Language & Platform Independence:
Objects built in different programming languages can interoperate over standard protocols.

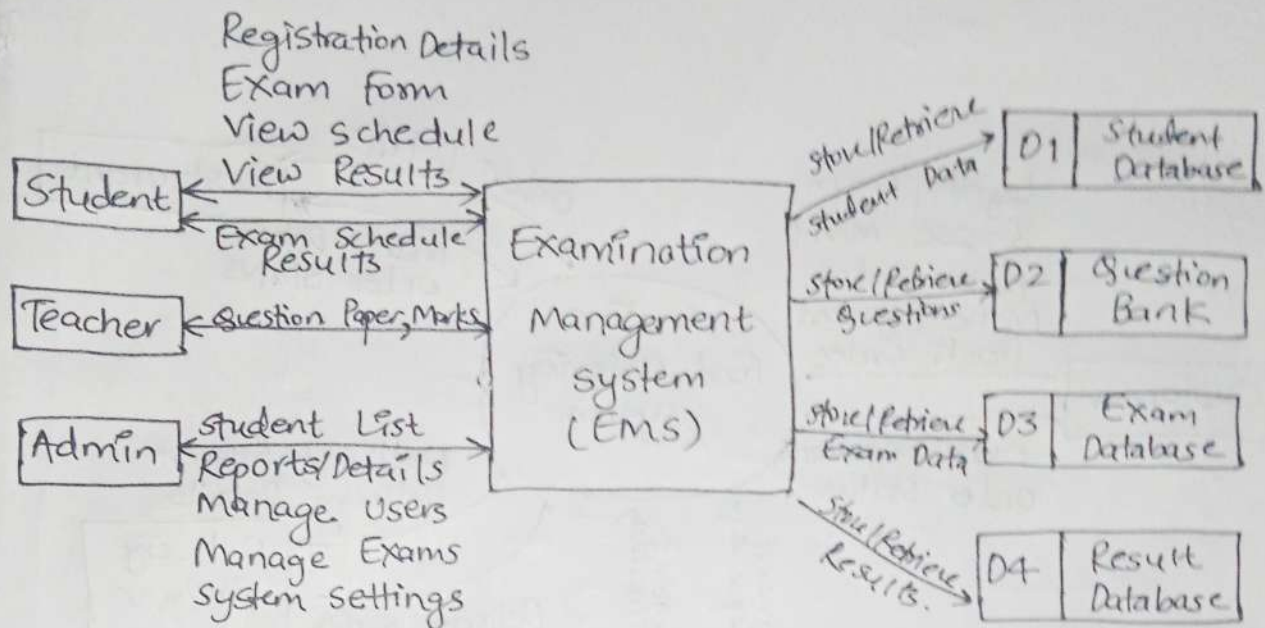
For example:

A credit-scoring application where a Loan Processing Service running on Linux calls a remote Risk Assessment Object hosted on a mainframe using gRPC or Remote Method Invocation (RMI). The loan Service executes

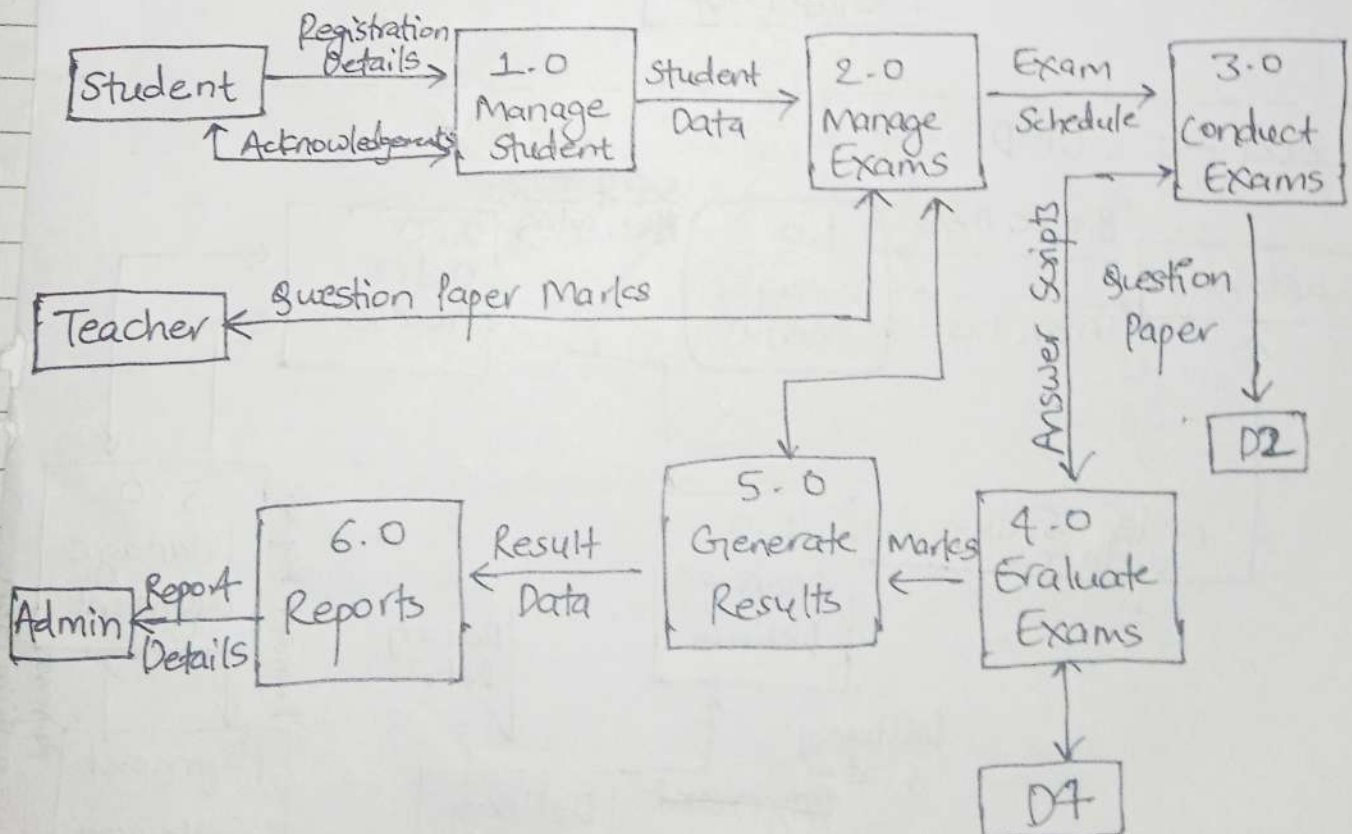
- `calculateRiskScore(userId)` seamlessly without managing low-level TCP/IP socket connections.

14. Design Level-0 and Level-1 DFD for the Examination Management System (EMS) case study.

Ans Level 0 DFD (Context Diagram)



Level-1 DFD

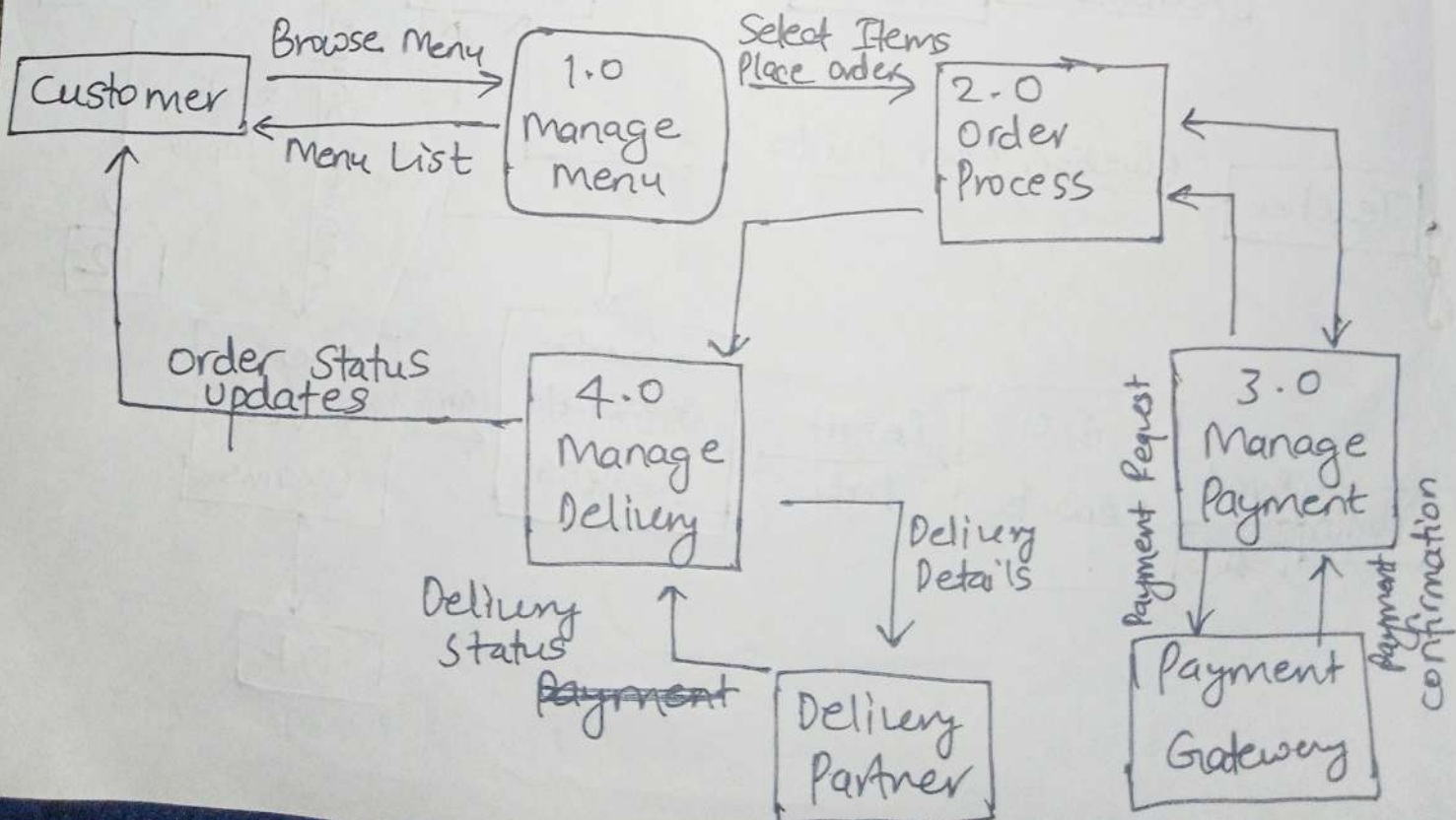


15. Develop a context diagram and Level 1 DFD for an online food ordering system.

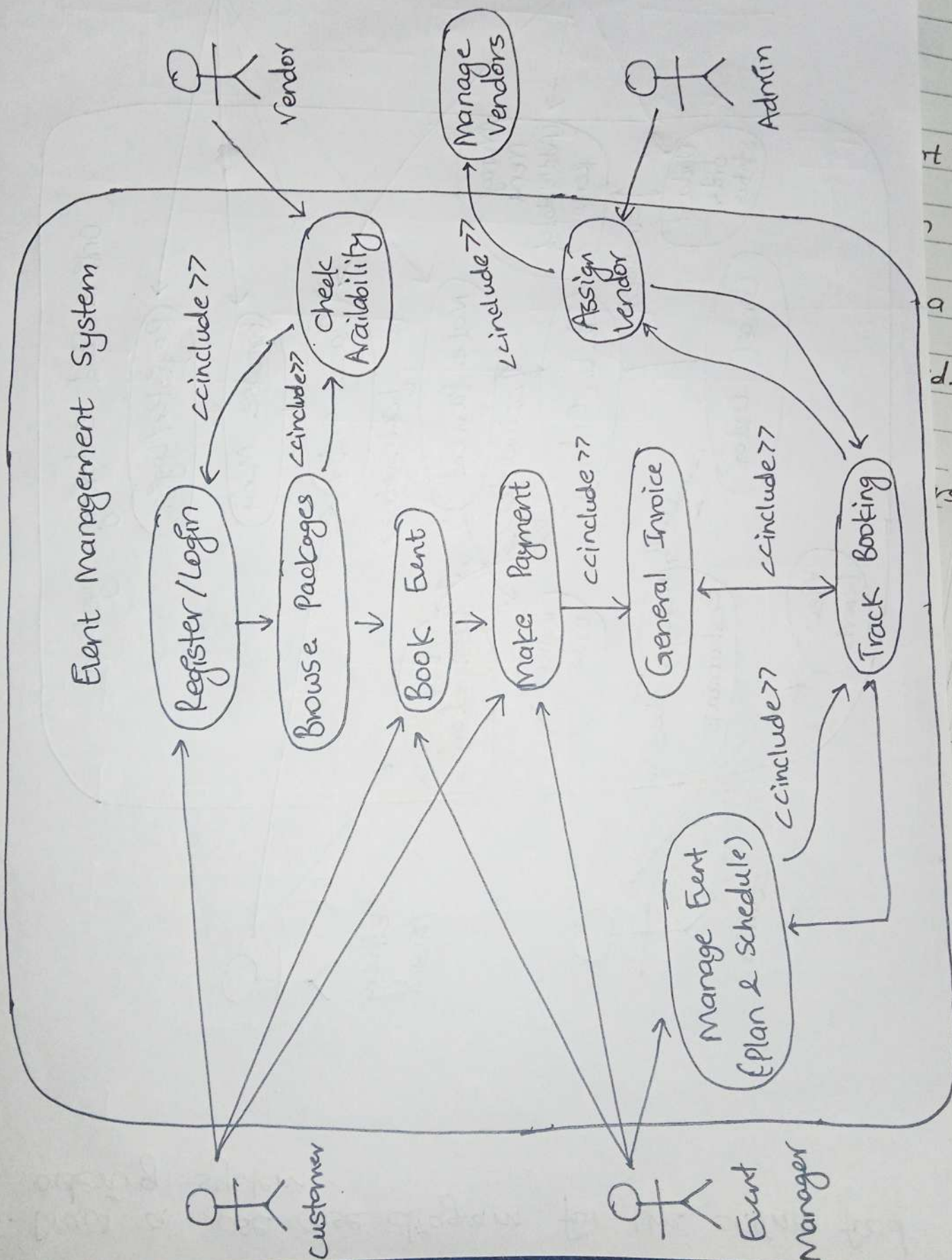
Ans Context Diagram (Level 0 DFD)



Level - 1 DFD



1.6. Prepare a use case diagram for the event management company scenario.



an 17. Draw a use case diagram for the online food ordering system.

